TypeScript Without Surprises: Smarter Error Handling with Effect

Nir Tamir

- Senior Frontend developer
- Loves open source and tooling
- nirtamir.com
- @NirTamir

TypeScript is great

TypeScript is great

```
const result = divide("hi", 2);
```

> Argument of type 'string' is not assignable to parameter of type 'number'.

TypeScript is great

```
const result = divide(1, 2); // number
```

```
const
result:
number
```

When the type system can’t help

```
const result = divide(4, 0); // Infinity
```

```
const
result:
number
```

Errors are typed as 'unknown'

```ts
try {
  const result = divide(4, 0);
} catch (error /* unknown */) {
  //
  //
  //
  //
  // What is this? Who knows! 🤷
}
```

The invisible throw problem

```
import { makeCoffee } from "./makeCoffee";

const coffee = makeCoffee(); // 💥 throws Error("MachineOutOfWaterError ☕")
```

```
function
makeCoffee():
Coffee
```

How do you handle an error you can’t see?

We're back to the JavaScript days

How we handle errors today

Custom error classes

```
class CustomError extends Error {
  _tag = "CustomError";
}
```

```
function doSomething() {
  if (Math.random() > 0.9) {
    throw new CustomError();
  }
  return "✅";
}
```

```
try {
  const result = doSomething(); // throws CustomError()
} catch (error /* unknown */) {
  if (error instanceof CustomError) {
    console.error(error);
  }
}
```

Return errors as Values

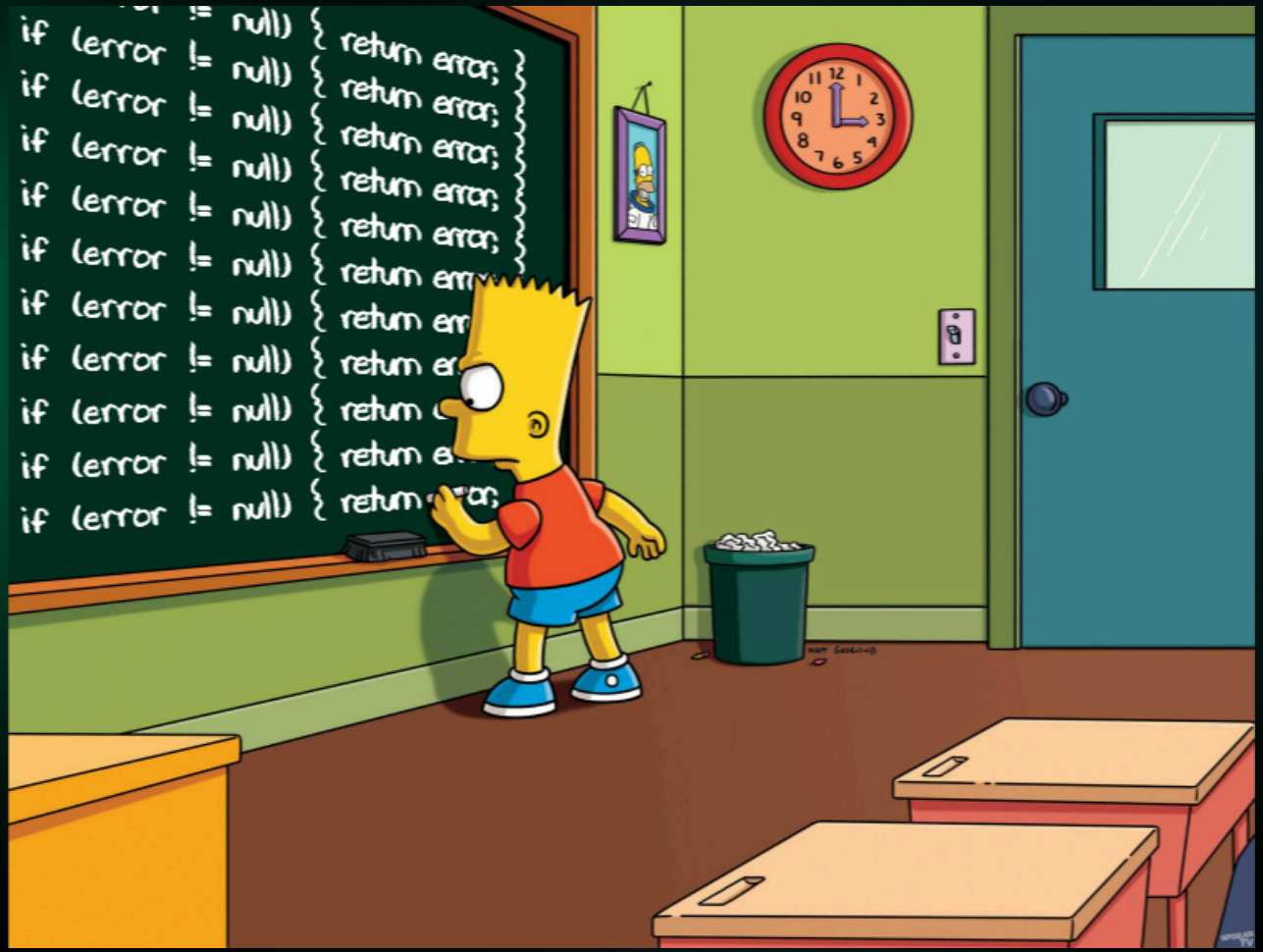
```ts
type Result<Data, Error> =
  | { data: Data; error: never }
  | { data: never; error: Error };
```

```ts
function doSomething(): Result<string, CustomError> {
  if (Math.random() > 0.9) {
    return { error: new CustomError() };
  }
  return { data: "✅" };
}
```

```ts
const result = doSomething();

if (result.error == null) {
  console.log(result.data); // string
} else {
  console.error(result.error); // CustomError
}
```

Wrapping and Unwrapping Gets Messy

```
function doSomething(): DoSomethingResult {
  const a = stepOne(1);
  if (a.error ≠ null) {
    return a;
  }

  const b = stepTwo(a.data);
  if (b.error ≠ null) {
    return b;
  }

  return b.data;
}
```


if (error != null) { return error;
if (error != null) { return error;
if (error != null) { return error;
if (error != null) { return error;
if (error != null) { return error;
if (error != null) { return
if (error != null) { return
if (error != null) { return
if (error != null) { return
if (error != null) { return

TypeScript is great

For the happy path

What we're missing

Errors in the type system

Effect

is a powerful TypeScript library designed to help developers easily create complex, synchronous, and asynchronous programs.

The Effect type

```
import type { Effect } from "effect";

//              ✅      🚨        📦
//
//              ▼       ▼         ▼
Effect.Effect<Success, Error, Requirements>;
```

Effect Values

```ts
import { Effect } from "effect";

const value = Effect.succeed(42); // Effect.Effect<number, never, never>

const error = Effect.fail("Oops"); // Effect.Effect<never, string, never>
```

Effects are blueprints

```
const program = Effect.succeed(42);
// This doesn't run anything yet!

const result = Effect.runSync(program);
// NOW it runs and we get 42
```

A Real Example

```ts
import { Effect, Data } from "effect";

class PaymentFailed extends Data.TaggedError("PaymentFailed")<{}> {}

//      ┌─── Effect<string, PaymentFailed, never>
//      ▼
const program = Effect.gen(function* () {
  if (isCreditExceeded()) {
    return yield* Effect.fail(new PaymentFailed());
  }
  return "payment-1234";
});
```

Error in the type

```
//                                ┌── Effect<string, PaymentFailed, never>
//                                ▼
const result = Effect.runSync(program);
// 💥 throws PaymentFailed
```

Error handled in the type

```
const safeProgram = program.pipe(
  Effect.catchTag("PaymentFailed", () ⇒ Effect.succeed("On the house 💸")),
); // Effect<string, never, never>

const result = Effect.runSync(safeProgram);
// string - Safe! ✅
```

The type guides us

```
// ❌ Before handling
const program: Effect<string, PaymentFailed, never>;
// Compiler says: "You have an unhandled error!"
```

Real programs have many failures

```
import { Effect, Data } from "effect";

class NetworkError extends Data.TaggedError("NetworkError")<{}> {}
class ValidationError extends Data.TaggedError("ValidationError")<{}> {}

//      ┌── Effect<User, NetworkError | ValidationError, never>
//      ▼
const program = Effect.gen(function* () {
  const user = { id: "123", name: "Alice" };

  // These are Effect-returning functions
  yield* validateUser(user); // might fail with ValidationError
  return yield* sendToServer(user); // might fail with NetworkError
});
```

Handling Multiple Errors

```
const safeProgram = program.pipe(
  Effect.catchTags({
    NetworkError: () ⇒ Effect.succeed(cachedUser),
    ValidationError: (error) ⇒ Effect.fail(new BadRequestError(error)),
  }),
); // Effect<User, BadRequestError, never>

const result = Effect.runSync(safeProgram);
// NetworkError and ValidationError are handled! ✅
```

Before Effect

```
async function program() {
  const data = await fetchData(); // might throw
  const parsed = parseData(data); // might throw
  return saveData(parsed); // might throw
}

// What can go wrong? Nobody knows! 🤷
```

After Effect

```
const program = Effect.gen(function* () {
  const data = yield* fetchData(); // Effect<Data, FetchError>
  const parsed = yield* parseData(data); // Effect<Parsed, ParseError>
  return yield* saveData(parsed); // Effect<Data, SaveError>
}); // Effect<Data, FetchError | ParseError | SaveError>

// The type tells us EXACTLY what can fail ✅
```

The difference

Let's see what changed

```
async function program() {
  const data = await fetchData();
  const parsed = parseData(data);
  return saveData(parsed);
}
```

We can use the type system to track **errors**, not only **success** values


GENERIC
EXCEPTION

From error tracking to recovery

We can recover, retry, and compose effects in many ways

Beyond Error Handling: Timeouts

Add a time limit to an effect, failing with timeout if exceeded

```
const pizza = orderDelivery();
const result = Effect.timeout(pizza, "1 second");
```

Beyond Error Handling: Retries

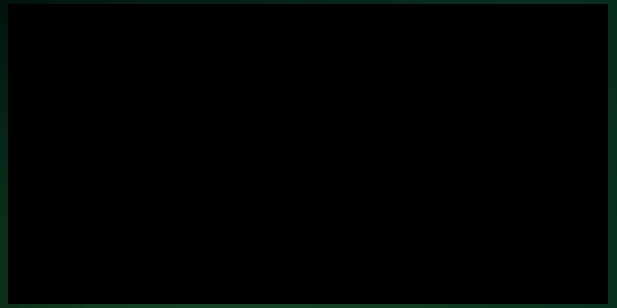
Run an effect repeatedly until it succeeds, ignoring errors

```
const swipeCard = swipeCard();
const result = Effect.eventually(swipeCard);
```

Beyond Error Handling: Retry with schedules

Retry an effect a fixed number of times

```
const wakeUp = attemptToWakeUp();
const snoozeSchedule = Schedule.intersect(
  Schedule.spaced("2 seconds"),
  Schedule.recurs(4),
);
const result = Effect.retry(wakeUp, snoozeSchedule);
```



Beyond Error Handling: Exponential backoff

Retry with exponential backoff

```
const park = attemptParallelPark();
const result = Effect.retry(park, Schedule.exponential("700 millis"));
```

More errors - better visibility

Dillon Mulroy λ @dillon_mulroy

typescript/javascript happy path blindness is real.

go through a critical code path in your application and note every single place an error can be thrown. are you handling each appropriately?

we did this with part of our domain renewal flow.

from 3 errors to 17

```
type RenewDomainError =
  | ApiError
  | StripePaymentError
  | StripePaymentMethodError;
```

Source Talk

Key Takeaways

- TypeScript is great for the happy path
- But errors are invisible in normal TypeScript
- Effect makes your errors visible
- Specific errors = reliable code

Thank you!

Slides: **nirtamir.com**

Questions? Find me after or at nirtamir.com

END

Learn More

Effect Resources

- Effect website - Official docs and guides
- Effect: Beginners Complete Getting Started - Free course
- Visual Effect - Interactive effect examples

Videos

- The Simple Secret Behind Effect's Power
- Effect: the unreadable library that captured my heart
- Dillon Mulroy - More errors, fewer problems

Community

- Effect Discord - Very active and helpful community